

MatchBook System Design Architecture

Lead Engineer Working Outline - MVP1

1. System Goal

MVP flow: Intake -> Structured Profiles -> Candidate Filtering -> Preliminary Match Scoring -> Verification -> Re-score with Verified Data -> Explainable Match -> Warm Introduction.

The architecture should stay lean enough for the eight-week MVP while leaving clean extension points for a future deal room, due diligence workflow, financing/lender integrations, and learning from match outcomes.

2. High-Level Architecture

Buyer / Seller / Admin Web App	
API / Backend - Auth + Business Logic	
Profile Service	Document Service
Structured Database	Secure Document Storage
Candidate Filtering - Hard Constraints	
Preliminary Matching Engine - Deterministic Scoring	
AI Extraction / Validation + Verification Status	
Update Structured Data -> Re-run Filters / Scoring	
AI Explanation + Human Review When Needed	
Mutual Interest -> Warm Introduction	

3. Intake and Structured Profiles

Seller profile

- Industry/business type and geography
- Revenue, SDE/EBITDA, asking price
- Owner involvement, employees, management structure
- Recurring revenue and customer concentration
- Financing preferences, transition period, closing timeline

Buyer profile

- Industries of interest and target geography
- Budget, minimum SDE, liquid capital and financing method
- Relevant experience and preferred ownership model
- Risk tolerance, transition requirements and deal preferences
- Explicit deal breakers / must-have constraints

Design principle: Intake should produce normalized structured profiles. The matching engine should not operate directly on a large free-form questionnaire.

4. Progressive Verification

Stage	Purpose	Access / Outcome
1. Marketplace Intake	Basic seller/business data; mostly self-reported	Enter matching pool and receive preliminary matches
2. Qualification / Verification	Core supporting documents such as P&L, tax return, and balance sheet	Validate key financial fields; re-score matches using supported data
3. Due Diligence	Bank statements, GL, contracts, leases, liens, and deeper records	Sensitive transaction workflow after serious mutual interest

5. Field-Level Verification

Avoid a single seller_verified = true/false flag. Track verification at the individual data-field level so preliminary matching can happen before every document is verified.

Field	Value	Status / Evidence
Revenue	\$1.40M	AI-supported - P&L;
SDE	\$310K	Document provided / pending review
Customer concentration	18%	Self-reported

Possible states: SELF_REPORTED, DOCUMENT_PROVIDED, AI_SUPPORTED, HUMAN_VERIFIED, DISCREPANCY.

Important: AI_SUPPORTED means the system found documentary evidence consistent with the field. It does not imply legal, audit, or lender-grade verification.

6. Candidate Filtering

Hard constraints should run before compatibility scoring. Support both **platform-level constraints** and **buyer-specific constraints**.

- Platform constraint example: buyer is not financially qualified for the transaction.
- Buyer constraint example: maximum purchase price cannot exceed \$1M.
- A field such as geography may be a hard constraint for one buyer but only a preference for another.

7. Matching Engine

Core formula: Compatibility Score = sum(Attribute Compatibility x Attribute Weight).

Attribute	Example Compatibility	Example Weight
Industry	1.00	25%
Location	1.00	15%
Price	1.00	20%
SDE	0.80	15%
Owner involvement	0.90	10%
Ownership model	1.00	10%
Risk / deal preferences	0.70	5%

8. AI's Role in Scoring

Do not ask an LLM to directly invent the final match percentage. AI should extract, interpret, and validate information; the scoring service should apply agreed rules and weights so identical inputs produce identical final scores.

Recommended flow: Intake/Documents -> AI extraction & interpretation -> Structured attributes/sub-scores -> Hard filters -> Deterministic weighted score -> AI-generated explanation.

AI is especially useful for fuzzy criteria such as adjacent-industry experience, interpreting open-text deal breakers, and extracting values from documents. Numeric and rule-based criteria should remain deterministic where possible.

9. AI Document Validation and Re-Scoring

Seller uploads document -> secure storage -> AI extracts structured financial data -> compare against seller claims -> supported field or discrepancy -> human review if confidence is low.

Example: Seller reports \$1.40M revenue and the P&L; shows \$1.39M: mark the revenue evidence as supported. If the seller reports \$310K SDE but supporting documents indicate materially less, flag the field for review.

Re-scoring loop: When validated or corrected values materially change a matching attribute, update the structured profile and re-run hard filters and deterministic scoring. A preliminary match can therefore become a verified match, change rank, or fail a hard constraint.

10. Compatibility vs. Verification

Keep these as separate concepts. **Compatibility** answers 'How well do these parties fit?' **Verification** answers 'How much confidence do we have in the underlying information?'

Example UI: **94% Compatibility** | Identity verified | Revenue supported | SDE verified | Customer concentration pending.

This avoids making a highly compatible pair appear less compatible simply because verification is incomplete. If verification changes an underlying value, the compatibility score itself is recalculated.

11. Explainability

After deterministic scoring, an AI explanation layer can translate the underlying evidence into a user-friendly explanation.

- Positive signals: industry alignment, budget fit, SDE fit, ownership model, financing alignment.
- Open risks: customer concentration pending, short lease term, unverified financial field, operational dependency.

12. Human in the Loop

Human review should be triggered by low AI confidence, significant financial discrepancies, conflicting high-risk information, or documents the system cannot reliably interpret. Human review should be a first-class system state, not an exception added later.

13. Warm Introduction

Strong compatibility + sufficient verification + mutual interest -> warm introduction.

Buyer expresses interest -> seller sees the appropriate buyer profile -> seller accepts -> MatchBook creates an introduction and enables the next conversation.

14. Core Data Model

- USER
- BUYER_PROFILE / BUYER_PREFERENCES
- SELLER_PROFILE / BUSINESS
- MATCH / MATCH_ATTRIBUTE_SCORE
- DOCUMENT / EXTRACTED_FIELD / VERIFICATION

- INTEREST / INTRODUCTION
- AUDIT_LOG

Important: Store the attribute-level reasons behind a match, not only match_score = 94.5. This supports explainability, debugging, evaluation, re-scoring, and future model improvement.

15. Security and Governance

- Role-based access: buyers, sellers, admins, and reviewers receive only appropriate permissions.
- Stage-based document access: a buyer may see a supported/verified metric without receiving the underlying tax return.
- Audit logging: who accessed a document, when, what AI processed it, what was extracted, and whether a human corrected it.
- Sensitive financial documents should remain separated from public/marketplace profile data.

16. MVP vs. Future Architecture

Build for MVP1	Design Extension Points For
Intake and profiles	Deal room / NDA
Database and hard filters	Secure diligence workspace
Matching engine + re-scoring	Lender marketplace / financing
AI extraction and validation	Closing workflow
Explainability and human review	Learning from accepted/declined/closed deals
Mutual interest / warm introduction	Deal-success prediction

Friday Architecture Discussion - Decisions to Make

- Confirm the logical architecture and MVP boundaries.
- Agree on how hard filters are defined: platform rules vs. user-defined must-haves.
- Define which compatibility sub-scores are deterministic and which may use AI interpretation.
- Agree that compatibility and verification are separate dimensions, while verified data can trigger re-scoring.
- Define verification states and human-review triggers.
- Agree on the minimum data model and service/module boundaries.
- Then select the concrete stack, cloud services, and engineering ownership.